

Envoxel: Live Environment Generation for Voxel Worlds

Ryan Hardesty Lewis
Massachusetts Institute of Technology (MIT)
Cambridge, Massachusetts, USA
rhl@mit.edu

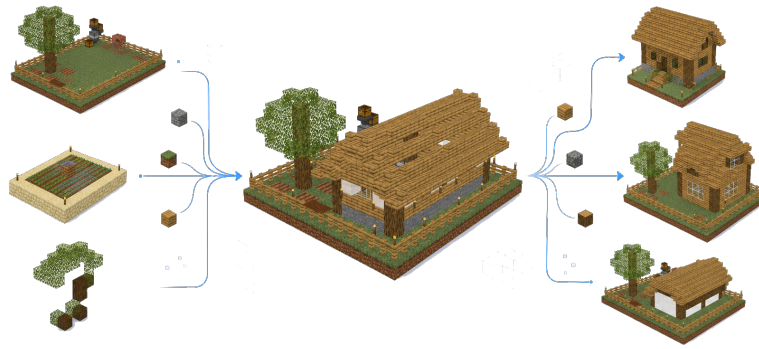


Figure 1: Envoxel lets users create with voxel world models, generating and editing whole voxel worlds in the browser.

Abstract

Envoxel is an interactive application for live voxel world generation. It lets a creator select or upload a voxel world, cut away part of it, stream autoregressive block predictions, compare generated variants, and export the accepted result back to a game-native representation. Unlike image, video, or mesh generation tools that require reconstruction before content can be edited in a voxel world, Envoxel keeps generated results as named blocks with positions, materials, and interiors. The Appy Hour experience emphasizes a short and inspectable creative loop: attendees can choose a seed, remove a region, watch candidate blocks appear in the browser, reject or accept alternatives, and leave with a clear sense of how sparse voxel generation can support hands-on world building.

CCS Concepts

• **Human-centered computing** → **User interface design**; • **Computing methodologies** → **Computer graphics**; *Machine learning*.

Keywords

voxel worlds, voxel generation, generative AI, interactive editing, game-native content creation

ACM Reference Format:

Ryan Hardesty Lewis. 2026. Envoxel: Live Environment Generation for Voxel Worlds. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Appy Hour (SIGGRAPH Appy Hour '26)*, July 19–23, 2026, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3799827.3816691>



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGGRAPH Appy Hour '26*, Los Angeles, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2550-0/26/07
<https://doi.org/10.1145/3799827.3816691>

1 Motivation

Every generative medium leaves a gap between what the model emits and what the user can touch. Image models hand back pixels, mesh models hand back triangles, and in both cases you end up editing *around* the model rather than *with* it. Voxel world models close that gap: the unit the model predicts, a block at a position with a material, is the very same unit a player has spent countless hours placing by hand. Generation and editing become the same.

Most prior work has explored this offline. VoxelCNN learns to place the next block in a partially built house in the order a human would [Chen et al. 2019], and World-GAN synthesizes whole worlds adversarially [Awiszus et al. 2021]; text-conditioned approaches such as MarioGPT push controllability further by steering generation with language [Sudhakaran et al. 2023]. Envoxel cares less about whether the model is right than whether you can make it right: you carve out a region, the model fills it, and nothing is committed until you say so. The author stays human.

2 Application Concept

Envoxel presents generation as live voxel world editing. A creator starts from a prepared seed, an uploaded world, or an empty workspace, then carves out whatever they want to rebuild, whether a wall, a house, a tower, or a whole quarter of the map, and asks Envoxel to fill the gap. The proposed blocks stay editable in the scene, and the result can be exported for inspection or play.

The app has four design goals. *Generation should be local and inspectable*: the user sees exactly which region is being changed, and the model runs on-device rather than in the cloud. *Outputs should stay symbolic*: blocks keep names, positions, and materials rather than becoming pixels or a baked mesh that cannot be easily or directly edited. *The user should control acceptance*: a bad generation can be thrown away without touching the original world. *Every step should be visible*: the process is transparent, and each workflow ends in something the user can rotate, edit, compare, or export. In this way, voxel world models become part of the design process.

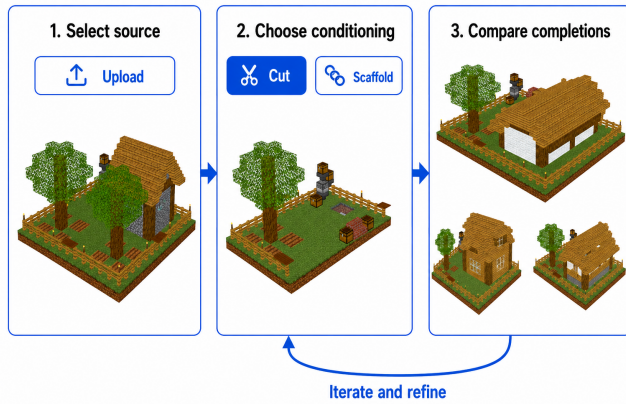


Figure 2: The live interaction loop: select or upload a source, carve out a region, stream generated variants, then compare and export an accepted voxel world.

3 System Loop

Figure 2 shows the loop the user sees. The browser editor holds the current scene, the selected source, the carved region, the prompt, and a few generation settings. When the user asks for a completion, the blocks around the cut are serialized as sparse tokens and handed to the model, which runs on-device. Running locally keeps the creative process in the user’s hands: there are no rate limits or usage caps between an idea and the next generation, the work stays private, and it all keeps running offline. It also keeps the loop interactive, since there is no server round-trip and completions return fast enough to feel immediate. Because the model emits blocks directly, its output is already the final editable representation; predictions land in the world as named blocks the user can change, with no conversion step between generating and editing.

The model does the hard part of generation; what Envoxel adds is the loop around it. The interesting question is not what a model produces on its own, but what a person can do with it: steer a completion toward an idea, and pull it back when it drifts. Every completion is reversible, so the user can keep a block, throw it away, or regenerate without ever touching the original world.

4 Demonstration Workflows

The Appy Hour demo runs as four short scripts. *Select or upload*: load a prepared world or drop in a small voxel sample. *Cut and complete*: remove a wall, a roof, or an interior and ask Envoxel to fill the gap. *Compare variants*: cut or scaffold the same area and generate several completions for it, so it is clear the output is a set of choices and not one fixed answer. *Export and reuse*: save the world as block data and drop it straight into a voxel game to play.

Each script is built around the user’s judgment. A completion can look good and still be wrong for the plan, so Envoxel shows its intermediate states and alternatives instead of hiding them behind one button. That makes even a bad generation useful: the user sees what changed, adjusts the prompt or the cut, and runs it again.



Figure 3: Envoxel running on mobile: a carved region (center) and two generated completions for it from text prompts, under different block palettes, spruce (left) and birch (right).

5 Audience Experience

Envoxel is meant to be used in a browser with the model on-device, so anyone can sit down, type a prompt, cut or scaffold a region, and generate a completion themselves. Because the model produces several completions, the user picks the one they like instead of being handed a single result.

The interface keeps the source, the region tools, the prompt, and the export button around the viewport, so the full cut-generate-edit-export loop is visible in one screen. A few seconds is enough for someone to go from an empty region to a finished voxel world they can export and play.

6 Limitations and Future Work

Envoxel is at its best on compact worlds, where a completion previews fast enough to stay in the loop. Bigger worlds need better scene partitioning and a cleaner way to edit several regions at once. Shape and palette are already under the user’s control: structure and palette conditioning let them pin down a completion’s form and block families and regenerate until it fits. The harder problem is consistency at scale, where a region can look right on its own yet clash with the world around it. Next steps include region labels, saved style presets, and multi-user review, so completions can become part of collaborative world-building.

Acknowledgments

We thank the players and builders who share their voxel worlds openly. Their creativity is the foundation that the future of voxel world creation will be built on.

References

- Maren Awiszus, Frederik Schubert, and Bodo Rosenhahn. 2021. World-GAN: a Generative Model for Minecraft Worlds. In *Proc. IEEE Conf. on Games (CoG)*.
- Zhuoyuan Chen, Demi Guo, Tong Xiao, Saining Xie, Xinlei Chen, Haonan Yu, Jonathan Gray, Kavya Srinet, Haoqi Fan, Jerry Ma, Charles R. Qi, Shubham Tulsiani, Arthur Szlam, and Larry Zitnick. 2019. Order-Aware Generative Modeling Using the 3D-Craft Dataset. In *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*.
- Shyam Sudhakaran, Miguel González-Duque, Matthias Freiberger, Claire Glanois, Elias Najarro, and Sebastian Risi. 2023. MarioGPT: Open-Ended Text2Level Generation through Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.